SOFTWARE DESIGN DOCUMENT (SDD)

Bharat Ballot

Secure Digital Election Management Platform

Version: 1.0
Prepared By: Team RawJet
Document Type: Software Design Document

Chapter 1 — Introduction

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the internal architecture, software design, implementation strategy, and technical organization of the Bharat Ballot platform. This document serves as a blueprint for developers, testers, maintainers, and future contributors by explaining how the system has been designed and implemented.

Unlike the Software Requirements Specification (SRS), which defines the functional and non-functional requirements of the system, this document focuses on the architectural decisions, software components, data organization, communication mechanisms, and deployment strategy used in the implementation of Bharat Ballot.

The document provides a comprehensive overview of the frontend applications, backend services, artificial intelligence module, database design, API architecture, authentication mechanisms, and cloud infrastructure that collectively enable secure digital election management.

1.2 Scope

This document covers the technical design of Bharat Ballot, including:

- Overall system architecture
- Frontend application design
- Backend architecture
- Artificial Intelligence service
- Database organization
- Authentication and authorization
- API architecture
- Deployment architecture
- Security mechanisms
- Design principles
- Future extensibility

It is intended to serve as a technical reference throughout the software lifecycle.

1.3 Intended Audience

This document is intended for:

- Software Developers
- System Architects
- Database Designers
- Testing Engineers
- Academic Evaluators
- Future Contributors
- DevOps Engineers

1.4 Design Goals

The primary design goals of Bharat Ballot are:

- Security
- Scalability
- Maintainability
- Reliability
- Performance
- Modularity
- Extensibility
- User Experience

The platform has been designed using a modular architecture so that each subsystem can evolve independently while maintaining seamless communication with other components.

1.5 Design Principles

The implementation of Bharat Ballot follows the following software engineering principles:

Modular Design

Each major functionality is implemented as an independent module.

Examples include:

- Authentication

- Elections
- Candidates
- Voters
- Constituencies
- AI Verification
- Results

Separation of Concerns

Frontend applications focus exclusively on presentation and user interaction.

Backend services manage:

- Business logic
- Authentication
- Database operations
- Validation

The AI service performs biometric verification independently.

Service-Oriented Architecture

The platform consists of independent services communicating through REST APIs.

Major services include:

- Citizen Portal
- Command Center
- Backend API
- AI Verification Service
- MongoDB Atlas
- Cloudinary

Reusability

Reusable software components are used throughout the project.

Examples include:

- Authentication middleware
- API utilities

- React components
- Database models
- Validation functions

Scalability

The architecture allows independent scaling of:

- Frontend
- Backend
- AI Service
- Database

1.6 Technology Overview

The Bharat Ballot platform has been implemented using modern web technologies.

Layer	Technology
Frontend	React + Vite
Backend	Node.js + Express
Database	MongoDB Atlas
ODM	Mongoose
Authentication	JWT + bcrypt
AI Service	Python + FastAPI
Face Recognition	InsightFace
Cloud Storage	Cloudinary
Deployment	Render
Version Control	Git + GitHub

1.7 Document Organization

This Software Design Document is organized as follows:

Chapter	Description
Chapter 1	Introduction

Chapter	Description
Chapter 2	Overall System Architecture
Chapter 3	Frontend Design
Chapter 4	Backend Design
Chapter 5	Database Design
Chapter 6	AI Service Design
Chapter 7	Authentication & Security Design
Chapter 8	API Design
Chapter 9	Deployment Design
Chapter 10	Design Decisions
Chapter 11	Future Enhancements

Chapter Summary

This chapter introduced the Software Design Document for Bharat Ballot by defining its purpose, scope, intended audience, design objectives, engineering principles, technology stack, and document organization. The following chapters describe the internal implementation of the platform, explaining how each subsystem collaborates to deliver secure, scalable, and reliable digital election management.

Chapter 2 — Overall System Architecture

2.1 Introduction

The Bharat Ballot platform follows a modular, service-oriented architecture designed to provide a secure, scalable, and maintainable digital election management system. Instead of implementing all functionalities within a single application, the platform separates responsibilities into independent software components that communicate through RESTful APIs.

This architecture enables each subsystem to perform specialized tasks while remaining loosely coupled with other components. Such an approach simplifies maintenance, improves scalability, and allows individual services to evolve independently without affecting the overall system.

The major architectural components include the Citizen Portal, Command Center, Backend API Server, Artificial Intelligence Face Verification Service, MongoDB Atlas database, and Cloudinary cloud storage.

2.2 Architectural Style

Bharat Ballot follows a **Service-Oriented Architecture (SOA)** combined with a **Client–Server Architecture**.

The system consists of multiple client applications that communicate with centralized backend services over secure HTTP-based REST APIs. The backend further integrates with external cloud services and an independent AI microservice responsible for biometric verification.

This architectural style provides the following advantages:

- Modular development
- Independent deployment of services
- High maintainability
- Improved scalability
- Better fault isolation
- Easier future enhancements

The separation of responsibilities ensures that changes made to one subsystem have minimal impact on the remaining components.

2.3 High-Level System Architecture

The Bharat Ballot platform is composed of the following major subsystems:

Citizen Portal

The Citizen Portal is a React-based frontend application that enables registered voters to:

- Authenticate using EPIC number and password.

- Complete biometric face verification.
- View active elections.
- Cast secure votes.
- View election results.
- Manage their profile.

The portal communicates exclusively with the Backend API Server through secure REST APIs.

Command Center

The Command Center serves as the administrative interface for Heads and Administrators.

Its responsibilities include:

- Election management
- Administrator management
- Candidate management
- Voter registration
- Constituency management
- Dashboard monitoring
- Result publication

The Command Center uses the same backend API while exposing functionality based on the authenticated user's role.

Backend API Server

The Backend API Server is the core processing unit of Bharat Ballot.

Developed using **Node.js** and **Express.js**, it is responsible for:

- Business logic execution
- Authentication
- Authorization
- Database operations
- Election lifecycle management
- Communication with AI services
- Communication with Cloudinary
- REST API management

All requests originating from the frontend applications are processed by the backend before interacting with the database or external services.

AI Face Verification Service

A dedicated Python-based AI microservice performs biometric identity verification.

The service uses:

- FastAPI
- InsightFace
- OpenCV
- ONNX Runtime

Its responsibilities include:

- Face enrollment
- Facial embedding generation
- Live face verification
- Cosine similarity comparison
- Verification response generation

Separating the AI service from the backend improves maintainability and enables future upgrades to the recognition model without affecting the rest of the platform.

MongoDB Atlas

MongoDB Atlas serves as the primary persistent storage system.

It stores:

- User accounts
- Elections
- Candidates
- Constituencies
- Voters
- Participation records
- Administrative data
- Election results

Mongoose ODM is used for schema management and database communication.

Cloudinary

Cloudinary is used for cloud-based image management.

Stored assets include:

- Voter photographs
- Candidate photographs
- Processed profile images

Cloudinary provides optimized image delivery while reducing storage requirements on the application server.

2.4 System Interaction

The interaction between major components follows the sequence below:

1. A user accesses either the Citizen Portal or Command Center.
2. The frontend sends requests to the Backend API Server.
3. The backend authenticates the user and validates authorization.
4. Business logic is executed based on the requested operation.
5. Database operations are performed using MongoDB Atlas.
6. If biometric verification is required, the backend communicates with the AI Face Verification Service.
7. If image storage or retrieval is required, the backend interacts with Cloudinary.
8. The backend returns the processed response to the frontend.
9. The frontend displays the result to the user.

This layered interaction minimizes direct dependencies between clients and external services.

2.5 Request Processing Workflow

Every user request follows a standardized processing pipeline:

Step 1: User Action
The user initiates an action through the web interface.

↓

Step 2: Frontend Validation
Basic validation is performed within the React application.

↓

Step 3: API Request
The frontend sends an authenticated REST request.

↓

Step 4: Authentication & Authorization
JWT tokens are validated, and user permissions are checked.

↓

Step 5: Business Logic Execution
The backend executes the requested operation.

↓

Step 6: External Service Communication (if required)

- MongoDB Atlas
- AI Service
- Cloudinary

↓

Step 7: Response Generation

↓

Step 8: Frontend Rendering

2.6 Layered Architecture

The Bharat Ballot platform follows a layered software architecture.

Presentation Layer

Implemented using React and Vite.

Responsibilities:

- User Interface
- Form validation
- Navigation
- Dashboard rendering
- API communication

Application Layer

Implemented using Node.js and Express.

Responsibilities:

- Business logic
- Authentication

- Authorization
- Request handling
- Validation
- Route management

Service Layer

Responsible for:

- AI Face Verification
- Image Management
- Cloud integrations

Data Layer

Implemented using MongoDB Atlas.

Responsibilities:

- Persistent storage
- Data retrieval
- Transaction management
- Relationship management

2.7 Architectural Advantages

The adopted architecture provides several benefits.

Security

- JWT Authentication
- Role-Based Access Control
- AI Face Verification
- Secure REST APIs

Scalability

Independent scaling of:

- Frontend applications
- Backend server

- AI microservice
- Database

Maintainability

Each module has a clearly defined responsibility, simplifying updates and debugging.

Extensibility

New modules such as notification systems, analytics, or mobile applications can be integrated without major architectural changes.

Reliability

The separation of services reduces the impact of component failures and improves overall system stability.

2.8 Architecture Diagram

Figure 2.1 – Overall System Architecture

(Insert the complete architecture diagram created in Draw.io here.)

The architecture diagram illustrates the interaction between the Citizen Portal, Command Center, Backend API Server, AI Face Verification Service, MongoDB Atlas, and Cloudinary. It provides a visual representation of the system's service-oriented architecture and the communication pathways between each component.

Chapter Summary

This chapter presented the overall architecture of the Bharat Ballot platform, describing its service-oriented design, major software components, layered architecture, request processing workflow, and interactions with external services. The adopted architecture ensures modularity, scalability, maintainability, and security while supporting the complete digital election lifecycle. It establishes the technical foundation for the detailed subsystem designs discussed in the subsequent chapters.